

Machine Learning Models Deep Dive & Comparison

Case Study 139: Soil Classification Project

A Complete Guide explaining WHAT, HOW, WHY, and WHEN each of the 5 ML Models is used in this project.

Summary Benchmark of Evaluated Algorithms

Algorithm	Model Family	Test Accuracy	Macro F1-Score	5-Fold Stratified CV	Preprocessing Needed?	Best For
1. Logistic Regression	Linear / Probabilistic	99.29%	99.29%	98.54 ± 0.86%	■ Mandatory (StandardScaler)	Fast, interpretable linear boundaries
2. Random Forest	Ensemble Bagging	99.11%	99.11%	98.50 ± 0.58%	■ Invariant (Raw works great)	Robust, non-linear, outlier resistant
3. Gradient Boosting	Ensemble Boosting	98.39%	98.40%	98.36 ± 0.51%	■ Invariant (Raw works great)	Sequential error correction
4. K-Nearest Neighbors (KNN)	Instance-Based Lazy	98.04%	98.03%	98.11 ± 0.54%	■ Mandatory (StandardScaler)	Local geometric clustering
5. Decision Tree	Non-Parametric Tree	95.89%	95.88%	95.68 ± 0.63%	■ Invariant (Raw works great)	Simple rule-based if-else splits

1. Logistic Regression (Multinomial Softmax)

What is it?

Despite its name containing "Regression", Logistic Regression is a **linear classification algorithm**. For multi-class problems (like our 8 soil types), it calculates the mathematical probability that an input soil measurement belongs to each of the 8 soil categories and picks the highest probability.

HOW does it work under the hood?

1. **Linear Score:** For each soil class k , the model computes a weighted score based on the 9 input features:
$$z_k = w_1 \cdot \text{pH} + w_2 \cdot \text{Nitrogen} + \dots + w_9 \cdot \text{Humidity} + b_k$$
2. **Softmax Transformation:** It turns these raw scores into probabilities between 0% and 100% such that the sum of all 8 probabilities equals 1.0 (100%):
$$P(\text{Soil Type} = k) = \frac{e^{z_k}}{\sum_{j=1}^8 e^{z_j}}$$
3. **Loss Minimization:** During training, it uses an optimization algorithm called **L-BFGS** to minimize Cross-Entropy Loss (error penalty when predictions are wrong).

WHY do we use it in this project?

- It provides a clean, fast linear baseline.
- When geochemical features are standardized with `_CODE_PH_0_`, soil boundaries become linearly separable in multi-dimensional space, enabling Logistic Regression to achieve an outstanding **99.29% accuracy**.

■ ■ WHEN should you use it?

- When you need fast inference (fractions of a millisecond).
- When you want true probabilistic outputs (e.g. "88% chance of Black Soil, 12% chance of Clayey Soil").
- When features have linear relationships with class log-odds.

■ ■ Hyperparameters used in our project:

- `max_iter=1000`: Allows the L-BFGS solver sufficient iterations to converge.
- `multi_class='multinomial'`: Solves all 8 classes simultaneously rather than training 8 separate binary models (One-vs-Rest).

2. K-Nearest Neighbors (KNN)

■ What is it?

KNN is an **instance-based, "lazy" learning algorithm**. It does not learn an explicit mathematical equation during training. Instead, it memorizes all 2,240 training soil samples. When a new soil sample is tested, it searches for the k closest neighboring samples in feature space and takes a vote.

■ ■ HOW does it work under the hood?

1. **Distance Calculation:** When a query sample x arrives, KNN calculates the straight-line Euclidean distance to every training sample:

$$d = \sqrt{(pH_1 - pH_2)^2 + (N_1 - N_2)^2 + \dots + (EC_1 - EC_2)^2}$$

2. **Finding the k Neighbors:** It selects the $k=5$ closest soil samples.
3. **Distance-Weighted Voting (`weights='distance'`):** Neighbors that are extremely close receive more voting weight ($\text{weight} = \frac{1}{\text{distance}}$) than neighbors further away.
4. **Output:** The majority vote determines the classified soil category.

■ WHY do we use it in this project?

- Soils with similar mineral compositions cluster together in physical space.
- It provides a purely geometric perspective on soil similarity.

■ ■ The Feature Scaling Trap (Case Study Q5):

- **Without Scaling (Raw):** Accuracy was only **92.50%** because Potassium (range $0-300$) dominated Euclidean distance, masking Electrical Conductivity (range $0-2$).
- **With StandardScaler:** Accuracy surged to **98.04% (+5.54% gain)**.

■ ■ WHEN should you use it?

- When the dataset size is small to moderate ($< 50,000$ rows).
- When data clusters naturally into geometric neighborhoods.

3. Decision Tree Classifier (CART)

■ What is it?

A Decision Tree is a non-parametric model that classifies soil by asking a sequence of **if-else questions** organized like an upside-down tree (Root $\rightarrow$ Branches $\rightarrow$ Leaves).

■ HOW does it work under the hood?

1. Splitting Criterion (Gini Impurity): At every node, the tree evaluates every possible split on all 9 features to find the split that produces the purest child nodes.

$$\text{Gini Impurity} = 1 - \sum_k p_k^2$$

A node with a Gini score of `_CODE_PH_0_` is 100% pure (all samples belong to one soil type).

2. Example Tree Rule:

- If Electrical Conductivity > 2.45 dS/m $\rightarrow$ **Saline Soil** (Gini = 0.0)
- Else if Organic Matter $> 4.8\%$ $\rightarrow$ **Peaty Soil** (Gini = 0.0)
- Else if Moisture $> 40\%$ $\rightarrow$ Check Potassium...

3. Leaf Node: When a sample reaches the bottom leaf, the majority class of that leaf becomes the prediction.

■ WHY do we use it in this project?

- High interpretability: Humans can easily read and draw the exact flowchart rules.
- Fast, non-linear classification without needing any data scaling.

■ WHEN should you use it?

- When human explainability is paramount.
- As a foundational building block for ensemble models (Random Forest).

4. Random Forest Classifier (Ensemble Bagging)

■ What is it?

Random Forest is an **Ensemble Learning** method that combines predictions from **120 independent Decision Trees** using **Bootstrap Aggregation (Bagging)** and random feature selection.

■ HOW does it work under the hood?

- 1. Bootstrapping:** 120 different training subsets are created by randomly sampling the 2,240 training records with replacement.
- 2. Feature Randomness:** When splitting a node in any tree, it only considers a random subset of $\sqrt{9} = 3$ features rather than all 9. This ensures that the 120 trees are decorrelated (they make different types of errors).
- 3. Parallel Training:** All 120 trees grow to deep levels independently.
- 4. Majority Voting:** When predicting a new soil sample, all 120 trees cast a vote. The soil class with the most votes wins!

$$\hat{y} = \text{mode}(\text{Tree}_1, \text{Tree}_2, \dots, \text{Tree}_{120})$$

■ WHY do we use it in this project?

- A single Decision Tree can easily overfit (achieved 95.89%). Random Forest eliminates overfitting by averaging 120 trees, achieving **99.11% accuracy** and **98.50% 5-Fold CV**.
- Automatically computes **Feature Importance** (Mean Decrease in Impurity).
- Completely scale-invariant (works identically on raw vs. scaled data).

■ ■ WHEN should you use it?

- The gold-standard default choice for tabular data with complex non-linear interactions.

5. Gradient Boosting Classifier (Sequential Boosting)

■ What is it?

Gradient Boosting is an **Ensemble Boosting** technique that builds trees **sequentially** (one after another). Unlike Random Forest where trees are independent, each new tree in Gradient Boosting specifically learns to correct the residual mistakes made by the previous trees.

■ ■ HOW does it work under the hood?

- 1. Initial Base Prediction:** Starts with a simple baseline guess (the average class log-odds).
- 2. Compute Residuals (Pseudo-Gradients):** For each sample, it calculates how much the current ensemble missed the true answer:

Residual = True Label - Predicted Probability

- 3. Fit New Tree to Residuals:** A shallow decision tree (**max_depth=5**) is trained to predict these residual errors.
- 4. Shrinkage / Learning Rate ($\eta = 0.1$):** The new tree's predictions are scaled down by a learning rate (**$\times 0.1$**) before being added to the ensemble, preventing aggressive overshooting:

$F_m(x) = F_{m-1}(x) + 0.1 \times h_m(x)$

- 5. Repeat for 120 Iterations:** The model sequentially reduces training loss step-by-step.

■ WHY do we use it in this project?

- Demonstrates the Boosting paradigm alongside Bagging (Random Forest), satisfying Case Study 139 requirement 4.
- Achieved **98.39% accuracy** and **98.36% 5-Fold CV**.

■ ■ WHEN should you use it?

- When maximum predictive precision is required on structured tabular data.
- When datasets have subtle, complex decision boundaries that benefit from focused iterative learning.

■ Head-to-Head Comparison Summary

Feature	Logistic Regression	K-Nearest Neighbors	Decision Tree	Random Forest	Gradient Boosting
Model Type	Linear / Parametric	Instance / Non-parametric	Tree / Non-parametric	Ensemble (Bagging)	Ensemble (Boosting)
Training Speed	■ Ultra-Fast (< 0.1s)	■ Instant (No training)	■ Very Fast (< 0.05s)	■ Moderate (~0.5s)	■ Slower (~2.5s)
Prediction Speed	■ Ultra-Fast (< 1ms)	■ Slower ($O(N \cdot M)$)	■ Ultra-Fast (< 1ms)	■ Fast (~5ms)	■ Fast (~5ms)
Requires Scaling?	■ Yes (StandardScaler)	■ Yes (Mandatory)	■ No	■ No	■ No
Handles Non-Linearity?	■ Poorly (needs kernel)	■ Excellent	■ Moderate	■ Outstanding	■ Outstanding

Project Accuracy	99.29%	98.04%	95.89%	99.11%	98.39%
Project Macro F1	99.29%	98.03%	95.88%	99.11%	98.40%
5-Fold CV Score	98.54% ± 0.86%	98.11% ± 0.54%	95.68% ± 0.63%	98.50% ± 0.58%	98.36% ± 0.51%